

H.S. 2nd Year — Computer Science and Applications (2026)

Solved Suggestive Practical Question Paper — Code: 35T CSCA (Pr)

Full Marks: 30 | Pass Marks: 12 | Time: Three hours

Q. No. 1 — Answer any one (10 marks) [All five options solved]

(a) Write a program in C++ to manage the Bank Account of a person.

```
#include <iostream>
using namespace std;
class BankAccount
{
    private:
        long accNo;
        char name[30];
        float balance;
    public:
        void createAccount()
        {
            cout << "Enter Account Number: "; cin >> accNo;
            cout << "Enter Account Holder Name: "; cin >> name;
            cout << "Enter Initial Balance: "; cin >> balance;
        }
        void deposit(float amt)
        {
            balance = balance + amt;
            cout << "Amount Deposited Successfully." << endl;
        }
        void withdraw(float amt)
        {
            if (amt > balance)
                cout << "Insufficient Balance!" << endl;
            else
            {
                balance = balance - amt;
                cout << "Amount Withdrawn Successfully." << endl;
            }
        }
        void display()
        {
            cout << "\nAccount No : " << accNo;
            cout << "\nName      : " << name;
            cout << "\nBalance   : " << balance << endl;
        }
};

int main()
{
    BankAccount b;
    int choice;
    float amt;
    b.createAccount();
    do
    {
        cout << "\n1.Deposit 2.Withdraw 3.Display 4.Exit\nEnter choice: ";
```

```

        cin >> choice;
        switch (choice)
        {
            case 1: cout << "Enter amount to deposit: "; cin >> amt; b.deposit(amt);
break;
            case 2: cout << "Enter amount to withdraw: "; cin >> amt;
b.withdraw(amt); break;
            case 3: b.display(); break;
        }
    } while (choice != 4);
    return 0;
}

```

Sample Output:

```

Enter Account Number: 101
Enter Account Holder Name: Rahim
Enter Initial Balance: 5000
1.Deposit 2.Withdraw 3.Display 4.Exit
Enter choice: 3
Account No : 101
Name       : Rahim
Balance    : 5000

```

(b) Write a program in C++ to show the polymorphism using function overloading.

```

#include <iostream>
using namespace std;
class Area
{
    public:
        int calculate(int side)           // for square
        {
            return side * side;
        }
        int calculate(int length, int breadth) // for rectangle
        {
            return length * breadth;
        }
        float calculate(float radius)       // for circle
        {
            return 3.14 * radius * radius;
        }
};
int main()
{
    Area a;
    cout << "Area of Square      : " << a.calculate(5) << endl;
    cout << "Area of Rectangle   : " << a.calculate(4, 6) << endl;
    cout << "Area of Circle      : " << a.calculate(3.5f) << endl;
    return 0;
}

```

Sample Output:

```

Area of Square      : 25
Area of Rectangle   : 24
Area of Circle      : 38.465

```

(c) Write a program in C++ to sort the numbers in an array using selection sorting.

```
#include <iostream>
using namespace std;
int main()
{
    int n, i, j, min_idx, temp;
    cout << "Enter number of elements: "; cin >> n;
    int a[n];
    cout << "Enter " << n << " elements:\n";
    for (i = 0; i < n; i++)
        cin >> a[i];
    for (i = 0; i < n - 1; i++)
    {
        min_idx = i;
        for (j = i + 1; j < n; j++)
        {
            if (a[j] < a[min_idx])
                min_idx = j;
        }
        temp = a[min_idx];
        a[min_idx] = a[i];
        a[i] = temp;
    }
    cout << "\nSorted array (ascending order):\n";
    for (i = 0; i < n; i++)
        cout << a[i] << " ";
    return 0;
}
```

Sample Output:

```
Enter number of elements: 5
Enter 5 elements:
40 10 30 5 20
Sorted array (ascending order):
5 10 20 30 40
```

(d) Write a program in C++ to show the Multilevel inheritance.

```
#include <iostream>
using namespace std;
class Person          // base class (grandparent)
{
    public:
        void personDetails()
        {
            cout << "This is a Person." << endl;
        }
};
class Employee : public Person    // derived from Person (parent)
{
    public:
        void empDetails()
        {
            cout << "This person is an Employee." << endl;
        }
};
```

```

class Manager : public Employee // derived from Employee (child)
{
    public:
        void managerDetails()
        {
            cout << "This Employee is a Manager." << endl;
        }
};
int main()
{
    Manager m;
    m.personDetails(); // inherited from Person
    m.empDetails();    // inherited from Employee
    m.managerDetails(); // own member
    return 0;
}

```

Sample Output:

```

This is a Person.
This person is an Employee.
This Employee is a Manager.

```

(e) Write a program in C++ to show different visibility modes.

```

#include <iostream>
using namespace std;
class Base
{
    private:
        int a; // accessible only within Base
    protected:
        int b; // accessible within Base and derived classes
    public:
        int c; // accessible everywhere
        void setValues()
        {
            a = 10; b = 20; c = 30;
        }
};
class Derived : public Base // public inheritance
{
    public:
        void show()
        {
            // a is NOT accessible here (private in Base)
            cout << "b (protected) = " << b << endl; // accessible
            cout << "c (public) = " << c << endl; // accessible
        }
};
int main()
{
    Derived d;
    d.setValues();
    d.show();
    cout << "c accessed from main() = " << d.c << endl; // public, so accessible
    // d.b and d.a are not accessible directly from main()
}

```

```
    return 0;
}
```

Sample Output:

```
b (protected) = 20
c (public)     = 30
c accessed from main() = 30
```

Explanation: 'private' members are accessible only inside the same class; 'protected' members are accessible inside the class and its derived classes; 'public' members are accessible from anywhere the object is visible, including outside the class.

Q. No. 2 — SQL Queries based on the given table (any three) [All five solved, 1×3=3]

Table structure used below: Employee (Empno, Employee_Name, Salary)

(a) Write SQL command to show all the records of the table.

```
SELECT * FROM Employee;
```

(b) Write the SQL command to show the salaries more than 20000.

```
SELECT * FROM Employee WHERE Salary > 20000;
```

(This will display the records of Jack, Robin and Priya.)

(c) Write a SQL command to show whose salary is the biggest.

```
SELECT * FROM Employee WHERE Salary = (SELECT MAX(Salary) FROM Employee);
```

(This will display the record of Robin, whose salary 25000 is the highest. Alternatively: SELECT Employee_Name, MAX(Salary) FROM Employee;)

(d) Write a SQL command to show the records of the employees whose names start with the letter 'P'.

```
SELECT * FROM Employee WHERE Employee_Name LIKE 'P%';
```

(This will display the records of Pradeep and Priya.)

(e) Write a SQL command to show the total salary of all employees.

```
SELECT SUM(Salary) AS Total_Salary FROM Employee;
```

(This will display the sum: 21000+25000+17500+16000+22700 = 102200.)

Q. No. 3 — Project Work (6 marks)

Ans: The Project Work is an individual practical assignment that must be developed by the student using C++ and/or a database (SQL), and is evaluated by the examiner during the practical exam. Since this component is based on original work carried out and submitted by the student (not a fixed 'question' with one correct answer), it cannot be 'solved' generically here. Suggested guidance for scoring full marks:

- Choose a simple, practical, real-life problem (e.g., Student Management System, Library Management System, Bank Account Management, Billing System, or Inventory Management).
- Include a title page, objective, source code (in C++/with a database back-end), sample input/output, and a brief conclusion.
- Make sure the program compiles and runs correctly, uses classes/objects (OOP concepts), and — if a database project — uses proper SQL queries (CREATE, INSERT, SELECT, UPDATE, DELETE).
- Keep the project neatly documented/handwritten in the practical file as per your teacher's/institution's format, and be ready to explain the logic of your code during viva.

Q. No. 4 — Practical File (6 marks)

Ans: The Practical File is a record/notebook maintained by the student throughout the year, containing all the practical programs performed in the Computer Science laboratory (C++ programs and SQL queries), each written with aim, source code, and output. As it is a file physically prepared and maintained by the student (and checked/signed by the teacher/lab in-charge), it cannot be generically 'solved' — marks are awarded based on completeness, neatness, correctness of programs, and regularity of entries. Suggested guidance:

- Include all programs done during the year (e.g., simple C++ programs, OOP programs — classes, inheritance, polymorphism, constructors; array/searching/sorting programs; and SQL/DBMS practical queries).
- For each program, write: Aim/Objective, Source Code, and Output (with sample input if any).
- Keep the file neat, page-numbered, and get it signed/verified by your subject teacher before the practical examination.

Q. No. 5 — Viva Voce (5 marks)

Ans: Viva voce is an oral examination in which the examiner asks questions based on the practical syllabus (C++ programming, OOP concepts, data structures, DBMS/SQL, and networking basics) to test the student's understanding. As this is a direct interactive question-answer session (not a written question), it cannot be pre-solved here. Suggested preparation tips:

- Be thorough with the basic theory concepts already covered in the theory paper — e.g., class, object, constructor, inheritance, polymorphism, pointer, array, stack, queue, DBMS, SQL commands, and networking terms (LAN, WAN, bandwidth, FTP, etc.).
- Be able to explain your own project work and every program in your practical file line by line — examiners commonly ask about the logic of code you have written.
- Practise explaining answers confidently and in simple language, and revise the 'Q. No. 1' short-answer style topics from the theory paper, as viva questions are usually drawn from similar basic concepts.